

Parallel Programming

Course summary



Contents

1	Introduction	4
1.1	Motivation	4
1.2	Parallelism and Concurrency	4
1.3	Additional definitions	4
1.4	Threads	4
1.4.1	Creating Threads	5
1.4.2	States	7
1.4.3	Critical Section	7
1.4.4	Wait and Notify	8
1.4.5	Useful Methods	9
2	Hardware Parallelism	9
2.1	Vectorization	9
2.2	Instruction Level Parallelism (ILP)	9

2.3	Pipelining	9
3	Parallelism	10
3.1	Theory	10
3.1.1	Granularity	10
3.1.2	Speedup	10
3.1.3	Amdahl's Law	11
3.1.4	Gustafson's Law	12
3.1.5	Task Graphs	12
3.1.6	State Space Diagram	13
3.1.7	Dependency Graph	14
3.2	Implementation	14
3.2.1	Fork/Join Framework	14
3.2.2	Executor Service	18
4	Concurrency	19
4.1	Terminology	19
4.2	Mutual Exclusion	19
4.2.1	Dekker's Algorithm	20
4.2.2	Peterson Lock	21
4.2.3	Filter Lock	22
4.2.4	Backery Lock	23
4.2.5	TAS and CAS	24
4.2.6	TATAS	25
4.2.7	Exponential Backoff	25
4.3	Synchronization	26
4.3.1	Semaphores	26
4.3.2	Rendezvous	26
4.3.3	Barriers	26
4.3.4	Conditions	28
4.4	Lock Granularity	28
4.4.1	Coarse Grained Locking	29
4.4.2	Fine Grained Locking	30
4.4.3	Optimistic Locking	31
4.4.4	Lazy Locking	32
4.4.5	Pros and Cons	33
4.5	Non Blocking Algorithms	33

4.5.1	ABA Problem	33
4.6	Consensus	34
5	Additional Topics	35
5.1	Message Passing Interface (MPI)	35
5.2	Transactional Memory	39
5.3	Histories	40
5.4	Sorting Networks	42

This document is a summary of the Parallel Programming course that is lectured in the 2nd semester of Computer Science at ETH Zurich. I created this document to summarize and repeat the key learnings of this course. It may contain errors. Franz Schwinn, 08.2022.

1 Introduction

1.1 Motivation

Traditionally software has been written for serial computation where instructions are executed sequentially one after another and on a single processor. Parallel computing on the other hand makes use of additional computing resources in order to increase speedup (and thus saving time and lowering costs) and also to solve larger and more complex problems.

1.2 Parallelism and Concurrency

We differentiate between parallelism and concurrency.

Definition

Parallelism: Parallelism is about doing multiple things at the same time. We use additional computing resources for solving a problem. Suppose we have a string of 100 characters and would like to know how many times the letter 'a' appears. Let there be 4 processors in our system. Instead of using one processor to iterate over all 100 characters, we make use of all 4 processors by giving each processor a string of length 25. At the end, the only work that is left to do is to sum up the 4 results we got from each processor.

Definition

Concurrency: Concurrency is about dealing with multiple things at the same time. The goal of concurrency is to efficiently manage access to a shared resource.

We cannot parallelize an application on a system only has one core / processor (equivalent in this course). Because that core is able to work on only one thread (defined later) at a time. So we cannot "deal with multiple things at a time". But we can achieve concurrency.

1.3 Additional definitions

<https://cgl.ethz.ch/teaching/parallelprog22/pages/terminology.html>

1.4 Threads

A thread is an independent unit of computation that executes a piece of code. A single processor can spawn multiple threads but it cannot work on more than one thread at the time (no parallelism). It can however switch between multiple threads very quickly (concurrency), which makes it appear as if the processor is executing work in parallel. If a system has multiple processors, then there can be multiple threads running simultaneously in the system, which allows us to benefit from parallelism.

1.4.1 Creating Threads

In Java, we can create a thread by implementing the interface `Runnable` or by creating a subclass of `Thread`.

```
1 public class MyThread extends Thread {
2     int attribute;
3     MyThread(int attribute) {
4         this.attribute = attribute;
5     }
6     // we have to overwrite the run method
7     public void run() {
8         // this code is executed by the thread
9         System.out.println(attribute);
10    }
11 }
12
13 public class MainCode {
14     public static void main(String[] args) {
15         for(int i = 0; i < 5; i++) {
16             Thread t = new MyThread(i);
17             // we start the thread by calling the start method
18             t.start();
19         }
20     }
21 }
```

Or alternatively:

```
1 public class MyThread implements Runnable {
2     int attribute;
3     MyThread(int attribute) {
4         this.attribute = attribute;
5     }
6     // we have to overwrite the run method
7     public void run() {
8         // this code is executed by the thread
9         System.out.println(attribute);
10    }
11 }
12
13 public class MainCode {
14     public static void main(String[] args) {
15         for(int i = 0; i < 5; i++) {
16             Thread t = new Thread(new MyThread(i));
17             // we start the thread by calling the start method
18             t.start();
19         }
20     }
21 }
```

We can also directly create the runnable while defining the thread:

```
1 public class MainCode {
2     public static void main(String[] args) {
3         Thread t = new Thread(new Runnable(){
4             public void run() {
5                 // this code is executed by the thread
6                 System.out.println("Hello world");
7             }
8         });
9         t.start();
10    }
11 }
```

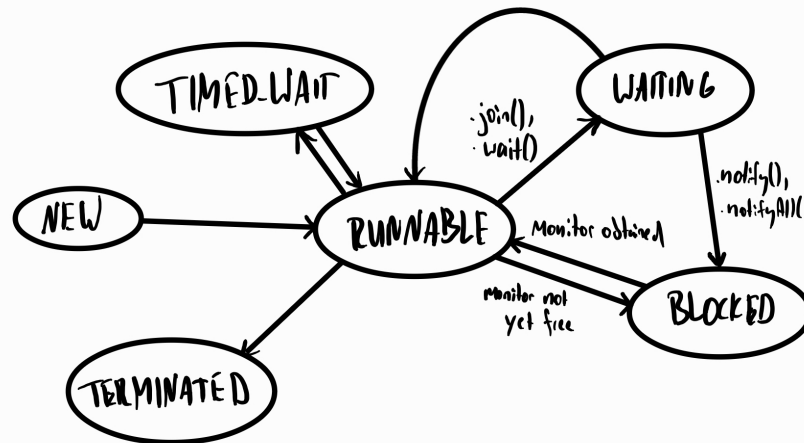
If we were to execute the main method that contains the for loop as shown above, we wouldn't be able to predict the output. The output would be different most of the times we run the code, because we don't have control over which threads get executed first. This nondeterministic behaviour means that we have to carefully design and test the program.

The `.start()` method starts the thread, whereas the `.run()` method only executed the code that is located in the `public void run()` method. Using the `.join()` method we can wait for a spawned thread to finish.

```
1 public class MyThread extends Thread {
2     int attribute;
3     MyThread(int attribute) {
4         this.attribute = attribute;
5     }
6     // we have to overwrite the run method
7     public void run() {
8         // this code is executed by the thread
9         System.out.println(attribute);
10    }
11 }
12
13 public class MainCode {
14     public static void main(String[] args) {
15         // we create Threads array
16         Thread[] threads = new Thread[5];
17         for(int i = 0; i < 5; i++) {
18             Thread t = new MyThread(i);
19             threads[i] = t;
20             // we start the thread by calling the start method
21             t.start();
22         }
23         for(int i = 0; i < 5; i++) {
24             // creating try-catch block because join can throw an
25             // InterruptedException
26             try {
27                 // we wait until all spawned threads are finished
28                 threads[i].join();
29             } catch (InterruptedException e) {}
30         }
31         System.out.println("Now all threads are done");
32    }
33 }
```

1.4.2 States

A thread always is in a certain state. The most important states are listed below (diagram is not complete).



- Non-existing: The thread has not yet been created.
- New: A thread enters this state once it has been created.
- Runnable: A thread enters the runnable state as soon as we call `start()`. The system can now start scheduling the thread.
- Blocked: This state is reached if the thread fails to acquire a lock. Once it successfully acquires the lock, it returns to the runnable state. We can also put the thread in blocked state by calling `join()` on it.
- Waiting: We can force a thread to enter the waiting state by calling `wait()`. If another thread calls `notify()` it may wake up and if another thread calls `notifyAll()` it for sure wakes up.
- Terminated: We can call `interrupt()` to terminate a thread. A thread also enters this state once its `run()` method has completed. Once the garbage collector finds the terminated thread, it will set its state to non-existent and thereby completing the cycle.

1.4.3 Critical Section

Imagine we create two different types of threads, one type responsible for incrementing a counter by 1 and the other type responsible for decrementing the counter by 1. If we were to set the counter to 0 at the beginning and run 10 threads of type "increment" and 10 threads of type "decrement" concurrently, we intuitively would expect the counter to be 0 at the end (because we perform the same number of increment operations as decrement operations). However, this must not necessarily be the case. The problem here is that the increment and decrement operations are not atomic, but rather a combination of three operations (read value, increment value, store value). In parallel execution, the counter could be updated by another thread after reading but before storing. Therefore we cannot expect the counter to be 0 at the end.

We can solve this problem by defining a critical section. There are multiple ways of implementing a critical section - one way is by using the `synchronized` keyword.

```
1 // we can synchronize an entire method
2 public synchronized void increment() {
3     this.counter++;
4 }
5
6 public void decrement() {
7     // or we can specify a block of code to be synchronized
8     synchronized(this) {
9         this.counter--;
10    }
11 }
```

Code in a critical section cannot be executed by two or more threads at the same time. Therefore we have guaranteed mutual exclusion, which means that we can be sure that at most one thread is executing the code in the critical section at a given time.

`synchronized` needs an object to synchronize on. We call this object the lock. Every object in Java has a lock. In both of the above examples (synchronization of the entire method and synchronization of the code section) we synchronized on the current object (`this`), so we used the current object as the lock.

If we face an exception during code execution the lock gets released.

```
1 public void error() {
2     synchronized(this) {
3         a(); // executes method a
4         divideByZero(); // throws exception and releases lock
5         b(); // b doesn't get executed
6     }
7 }
```

In general we want to keep the critical sections as small as possible in order to minimize the sequential part of the program and to maximize the parallelizable part of the program.

1.4.4 Wait and Notify

We can picture the idea of wait and notify as follows. Imagine a dog sitting in front of its bowl and waiting for it to be filled with food. As long as the bowl is empty, the dog waits. But after we put food into the bowl, we say "Enjoy!" to the dog (and thereby notifying it), and the dog enthusiastically starts to eat.

```
1 // dog
2 synchronized(bowl) {
3     while(bowl.isEmpty()) {
4         // thread gets to waiting state
5         bowl.wait();
6     }
7     eat();
8 }
9
10 // human
11 synchronized(bowl) {
12     bowl.addFood();
13     // wakes up all waiting threads
14     bowl.notifyAll();
15 }
```

`wait()` releases the lock and the thread enters into waiting state. `notifyAll()` wakes up all waiting threads and they compete non-deterministically for the lock. `notify()` wakes up the highest priority thread, which could result in low priority threads to never wake up and thus starve.

We placed the `bow1.isEmpty()` condition inside a while loop because of the following two reasons:

- `notifyAll()` wakes up all threads, even those whose entry condition has not yet been fulfilled.
- The Java virtual machine can randomly wake up threads without us having called `notify()` / `notifyAll()` (spurious wake ups). We don't want the thread to continue because the entry condition might not be fulfilled yet.

```
1 synchronized(lock) {  
2     while(<entry condition does not hold>) {  
3         lock.wait();  
4     }  
5     // entry condition is fulfilled  
6     // lock.notifyAll(); to notify all waiting threads  
7 }
```

1.4.5 Useful Methods

```
1 Thread t = Thread.currentThread(); // gets the current thread object  
2  
3 int thread_id = t.getId(); // gets thread id of thread t  
4  
5 t.setName("Some_Name"); // sets the name (not id) of thread t  
6  
7 t.setPriority(Thread.MAX_PRIORITY); // updates the thread's priority (JVM may or  
   // may not follow these priorities)  
8  
9 if(t.getState() == State.TERMINATED) {} // check if thread t is terminated
```

2 Hardware Parallelism

2.1 Vectorization

We use special hardware instructions to compute parallel actions such as adding two vectors componentwise. We cannot control vectorization of our code in Java and therefore have to trust the compiler and the JVM to do this for us.

2.2 Instruction Level Parallelism (ILP)

Instructions that are independent of each other can be calculated in parallel. Combined with speculative execution or Out-Of-Order execution, a speedup can be achieved. Here as well we cannot control ILP from within our Java code.

2.3 Pipelining

The idea of pipelining is to arrange hardware elements of the CPU such that the overall performance is increased. A sandwich factory is a real life example of a pipelined operation. There are 3 stages every sandwich has to pass: toasting of the bread, inserting peanut butter between two slices of bread, and packaging the sandwich.

Definition

Throughput: Amount of output that a pipeline produces per time unit (larger means better).

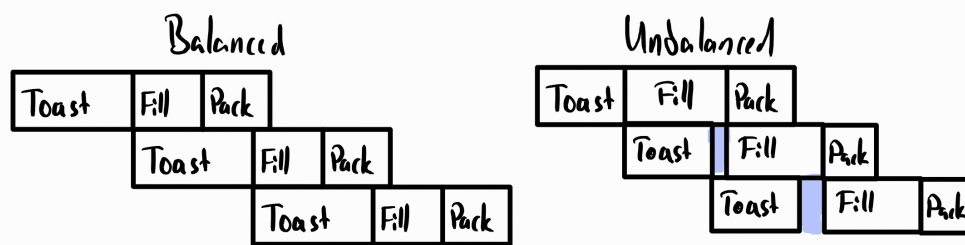
Bound on throughput: $\frac{1}{\max(\text{computationtime}(\forall \text{stages}))}$

Definition

Latency: Time it takes for one computation to finish. If the pipeline is balanced, the latency is constant over time and is equivalent to the sum of the computation time of all stages that are required.

Definition

Balanced: A pipeline is balanced $\Leftrightarrow$ the length of the first stage is greater than or equal to the maximum length of all other stages. The latency of a balanced pipeline stays constant over time, while the latency of an unbalanced pipeline goes to infinity.

**Useful**

Lead-In: Time until all the stages are busy

Full utilization: All stages are busy

Lead-Out: Time it takes from full utilization to all stages being free

3 Parallelism

3.1 Theory

3.1.1 Granularity

As a programmer we have to think about up to which level we want to split the problem into smaller parallelizable tasks in order to achieve the best performance. If we choose the granularity too low we don't fully utilize the available computing resources. If we choose the granularity too high we have too much parallelization / scheduling overhead. Ideally we choose the task size to be as small as possible but significantly bigger than the scheduling overhead.

3.1.2 Speedup

Definition

Sequential execution time: T_1 (time it takes for the program to finish on a single processor)

Parallel execution time: T_p (time it takes for the program to finish on p processors)

T_p can be bounded as follows:

$$\frac{T_1}{p} \leq T_p \leq \frac{T_1}{p} + T_\infty$$

Definition

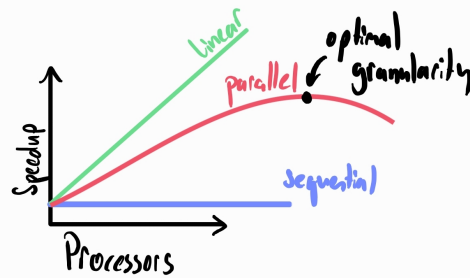
Speedup: $S_p = \frac{T_1}{T_p}$ (speedup on p processors is the sequential time T_1 divided by the time it takes for the program to finish using p processors).

Efficiency (relative speedup): $\frac{S_p}{p}$

Let us assume that a fully parallelizable program is executed on $p = 3$ threads. Then $T_1 = 1$ and $T_p = T_3 = \frac{1}{3}$. Then the speedup would be $S_p = \frac{T_1}{T_p} = S_3 = \frac{T_1}{T_3} = \frac{1}{\frac{1}{3}} = 3$. In this case we have a linear speedup of $S_p = p$ (ideal). But in reality we would never achieve an ideal speedup because

- often times programs have a sequential part that cannot be parallelized
- we have to consider the parallelization overhead (creating threads, context switching, ...)

That is why $S_p < p$ holds.



Using Amdahl's and Gustavson's Laws we can calculate a bound of the expected speedup.

3.1.3 Amdahl's Law**Theorem**

Amdahl's law: Assume a program with serial fraction f of the total work is executed on p processors. Then the speedup is:

$$S_p < \frac{1}{f + \frac{1-f}{p}}$$

Intuition: The time it takes to run the program on one processor is equivalent to the time it takes to run the sequential part of the program on one processor plus the time it takes to run the parallelizable part of the program on one processor.

$$T_1 = T_{ser} + T_{par}$$

The time it takes to run the program on p processors is equal to the time it takes to run the sequential part of the program plus the time it takes to run the parallelizable part of the program on p processors (actually greater than because of parallelization overhead).

$$T_p > T_{ser} + \frac{T_{par}}{p}$$

Plugging this into the definition of speedup:

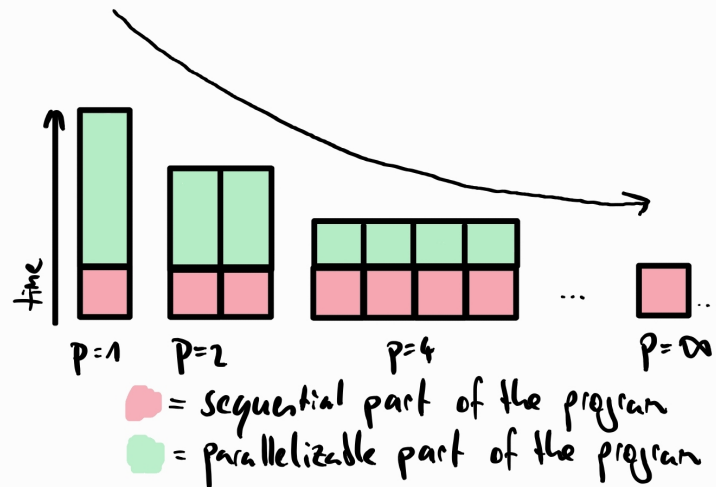
$$S_p = \frac{T_1}{T_p} < \frac{T_{ser} + T_{par}}{T_{ser} + \frac{T_{par}}{p}}$$

If we define f as the sequential fraction of the total work:

$$T_{ser} = f * T_1, T_{par} = (1 - f) * T_1$$

We get:

$$S_p < \frac{1}{f + \frac{1-f}{p}}$$



Amdahl's law shows that reducing the sequential part of a program even a tiny bit can result in a high speedup gain if executed on many processors.

3.1.4 Gustafson's Law

While Amdahl's law focuses on a fixed problem size that we want to compute in the least amount of time possible, Gustafson's law takes the other approach. We are interested in the maximum problem size that we can compute in a fixed amount of time. For instance, in a video game we would be interested in computing the highest possible resolution (or the best possible graphics) in $\frac{1}{60}$ seconds (duration of one frame in a 60 fps game).

Theorem

Gustafson's law: Assume a program with serial fraction f of the total work is executed on p processors. Then the speedup is:

$$S_p < f + p(1 - f)$$

3.1.5 Task Graphs

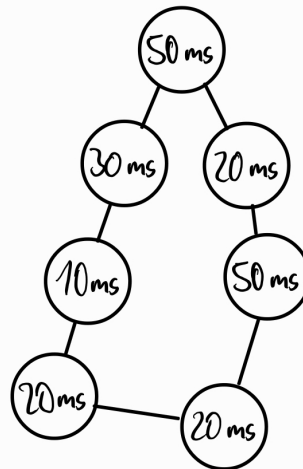
Task graphs allow us to visualize the sequential and parallelizable parts of a program and the dependencies between them. They also allow us to make useful statements about the parallelizability of the program. Task graphs are directed acyclic graphs. Consider this senseless example:

```

1 a = work_a() // takes 50 ms
2 b = work_b(a) // takes 30 ms
3 c = work_c(a) // takes 20 ms
4 d = work_d(b) // takes 10 ms
5 e = work_e(d) // takes 20 ms
6 f = work_f(c) // takes 50 ms
7 g = work_g(e, f) // takes 20 ms

```

The corresponding task graph would look like this:



Useful

T_1 : Time it takes to run the program on one processor (total work) - sum of cost of every node

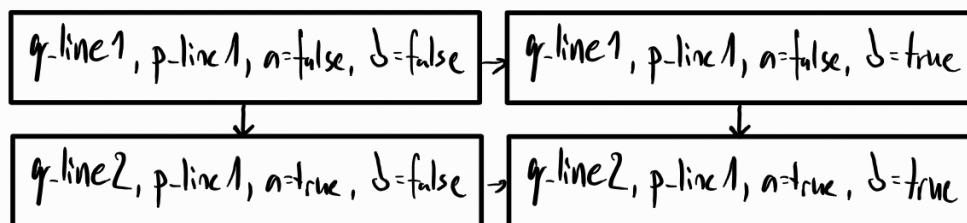
T_∞ : Time it takes to run the program on infinitely many processors - sum of cost of all nodes on the critical path (path with maximum cost from the root to a sink of the graph)

$\frac{T_1}{T_\infty}$: Parallelism

In the above example, T_1 is 200 ms (sum of all costs) and T_∞ is 140 (sum of all costs on critical path). Parallelism is $\frac{200}{140} = 1.42 \left(\frac{T_1}{T_\infty} \right)$ which gives us an upper bound for the speedup we can expect.

3.1.6 State Space Diagram

Another tool to analyze the behaviour of a parallel program is the state space diagram. This is a diagram that lists all possible states and transitions between them.



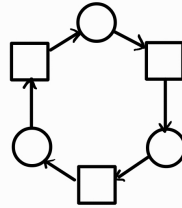
If we can reach a state where both processes are in their critical section, we have no mutual exclusion. If we can reach a state (that is not the final state) that has no outgoing edge, we have a deadlock. If we never reach the final state, we have starvation.

3.1.7 Dependency Graph

We can visualize processes, resources and its relation using dependency graphs. In this example, circles (○) represent processes, squares (□) represent resources, and their relation is:

$$\begin{aligned}\square &\rightarrow \bigcirc = \text{"has"} \\ \bigcirc &\rightarrow \square = \text{"wants"}\end{aligned}$$

A cycle in the dependency graph results in a deadlock (see example of dining philosophers problem below).



3.2 Implementation

3.2.1 Fork/Join Framework

Fork/Join is a framework that helps us achieving parallelization by using the divide and conquer approach. As mentioned in the beginning, we are interested in splitting the total computation into smaller tasks that can be assigned to multiple threads. The general structure of Fork/Join looks as follows:

```

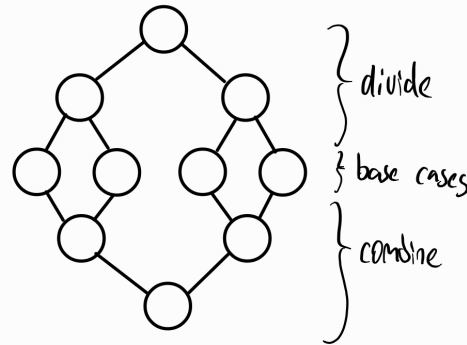
1  // this code returns the greatest sum of three consecutive elements
2  class MaxTriple extends RecursiveTask<Integer> { // 1.
3      int[] input;
4      int start;
5      int length;
6
7      public MaxTriple(int[] input, int start, int length) {
8          this.input = input;
9          this.start = start;
10         this.length = length;
11     }
12
13     public protected Integer compute() { // 2.
14         if(length == 1) { // 3.
15             if(start >= length-2) {
16                 return 0;
17             }
18             return input[start]+input[start+1]+input[start+2];
19         }
20         MaxTriple left = new MaxTriple(input, start, length/2); // 4.
21         MaxTriple right = new MaxTriple(input, start+length/2, length-length/2);
22         left.fork(); // 5.
23         return Math.max(right.compute() + left.join()) // 6.
24     }
25 }
26
27 ForkJoinPool f = new ForkJoinPool(); // 7.
28 int result = f.invoke(new MaxTriple({4,10,1,5,9,3,5,2}, 0, 8)); // 8.

```

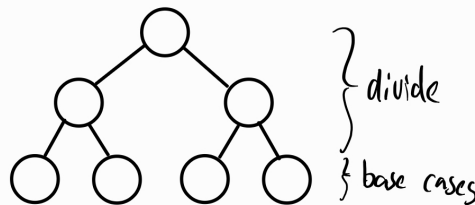
1. We have to give the Fork/Join pool an object that extends from `RecursiveAction<T>` (in case we expect some return value) or of type `RecursiveTask` (in case we don't expect some return value and just want to trigger an action).
2. Next we have to override the `compute()` method (analogously to the `run()` method in threads). We can insert our logic here.
3. We need to define a base case. The base case marks the point where we no longer divide the initial problem into smaller pieces but instead perform and return the expected work. In this example we enter the base case as soon as the thread only has one element to consider (`length == 1`). But usually we would implement a sequential cutoff which can be at around 500-1000 (greatly depends on the calculation) at which point we would solve the problem sequentially.
4. Since we want to divide the problem, we create two `MaxTriple` objects, each receiving one half of the initial input. The object responsible for the left half gets all elements from the interval of length `length/2` starting from `start` (thus without the element at position `start+length/2`) and the object responsible for the right half gets all remaining elements (elements in interval of length `length-length/2` starting from `start+length/2`). In other problems it may be that we don't receive the `length` attribute but instead a `to` attribute (which means that the element at index `to` is included in the interval). Then we need to adjust the splitting a little bit: `MaxTriple left = new MaxTriple(input, start, to/2);` and `MaxTriple right = new MaxTriple(input, start+to/2+1, to);`. So we need to be careful about whether or not we want to include the last element.
5. Now we call `fork()` on one of the objects, which results in a new thread being created. This newly created thread will execute its `compute()` method and potentially start more threads itself.
6. In this example we are interested in the maximum value of three consecutive elements. Therefore we keep returning the highest result we get. Instead of calling `right.fork()` we call `right.compute()` in order to keep the current thread busy. `compute()` will (just like `run()`) not spawn a new thread but instead executes the `compute()` method. We need to call `join()` on the left object in order to retrieve its result. Usually this is the place where we combine the results.
7. In order to start the whole process, we first create a Fork/Join pool. `ForkJoinPool(int n)` takes the parallelism level (tells Fork/Join how many worker threads to use). If we don't specify the number it will be equal to the amount of unused processors available in the system.
8. Using `invoke()` we can start the Fork/Join process. As an argument we pass an instance of our `RecursiveTask<T>` or `RecursiveAction` object. `invoke()` waits until the result is returned. If we want to perform some other work while Fork/Join is busy, we can use `submit()`. Instead of receiving a `<T>` (in this example an `Integer`) we would get a `Future<T>` object. But then we have to explicitly ask for the result by calling `Future.get()`.

There are several categories of problems that we can solve using Fork/Join.

- **Reduce:** A reduction is implemented when we are interested in a single answer using an associative operator. However, results of this type don't have to be single numbers or strings - they can also be arrays or objects with multiple fields. Examples of this type of problem are: highest sum of consecutive elements, number of occurrences of an element, maximum / minimum element, and so on.



- **Map:** Problems of type "map" don't return us a single result, but require us to operate on each element of the input independently to create an output of same size. Examples of this type of problems are: set every element of an array to the sum of all previous elements, multiply every element with some factor, and so on.



- **Pack:** Problems of this type are more sophisticated. They ask for an output containing only elements such that a certain function $f(element)$ is true. Examples are: which elements are greater than 10, which elements are even, and so on. These types of problems are more complex to solve. We first compute a bit-vector that holds 0 for elements that don't satisfy the function and 1 for elements that satisfy the function. Then we create the prefix sum on that bit-vector so that every element sees which index in the output array it should occupy.

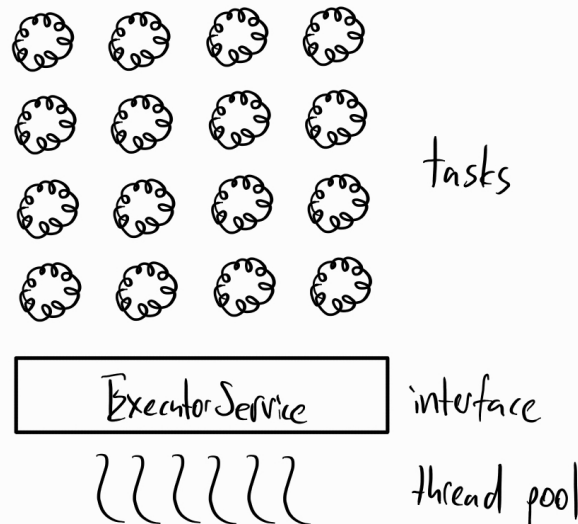
problem : which elements are greater than 10?
input : [17, 4, 6, 8, 11, 5, 13, 19, 0, 24]
bit-vector : [1, 0, 0, 0, 1, 0, 1, 1, 0, 1]
prefix sum of bit-vector : [1, 1, 1, 1, 2, 2, 3, 4, 4, 5]
map to produce output : [17, 11, 13, 19, 24]

Below is an implementation of maximum sub array sum.

```
1 public class MaxSubArraySum extends RecursiveAction {
2     int[] input;
3     int start;
4     int length;
5
6     // additional attributes required
7     int left;
8     int right;
9     int cross;
10    int sum;
11
12    public MaxSubArraySum(int[] input, int start, int length) {
13        this.input = input;
14        this.start = start;
15        this.length = length;
16    }
17
18    protected void compute() {
19        if(length == 1) {
20            this.left = input[start];
21            this.right = input[start];
22            this.sum = input[start];
23            this.cross = input[start];
24            return
25        }
26        MaxSubArraySum l = new MaxSubArraySum(input, start, length/2);
27        MaxSubArraySum r = new MaxSubArraySum(input, start+length/2, length-length
28            /2);
29        l.fork();
30        r.compute();
31        l.join();
32        this.left = Math.max(l.left, l.sum + r.left);
33        this.right = Math.max(r.right, r.sum + l.right);
34        this.sum = r.sum + l.sum;
35        this.cross = Math.max(r.left + l.right, Math.max(r.cross, l.cross));
36        return;
37    }
38
39    int[] input = {-2, -3, 4, -1, -2, 1, 5, -3};
40    MaxSubArraySum result = new MaxSubArraySum(input, 0, input.length);
41    ForkJoinPool p = new ForkJoinPool();
42    p.invoke(result);
43    System.out.println(result.cross); // 7
44    System.out.println(result.left); // 2
45    System.out.println(result.right); // 4
46    System.out.println(result.sum); // -1
47    System.out.println(Math.max(result.cross, Math.max(result.left, Math.max(result.
48        right, result.sum)))); // 7
```

3.2.2 Executor Service

Fork/Join is great if we have a regular workload and know how many threads we want to schedule at which point of time. However, there could be a situation where we have an irregular workload - a situation where we cannot make sure to split the work fairly between threads. This is where Executor Service comes into play. Instead of scheduling threads on tasks, we are interested in scheduling tasks on threads.



The idea behind Executor Service is that we focus on generating and submitting tasks while leaving the allocation of threads to tasks to an interface.

```

1  class PrintText implements Runnable { // 1.
2      int task;
3
4      public PrintText(int task) {
5          this.task = task;
6      }
7
8      public void run() { // 2.
9          System.out.println("Task " + task + " is executed on " + Thread.
10             currentThread().getId());
11      }
12  }
13
14  int num_tasks = 1000;
15  ExecutorService e = new Executors.newFixedThreadPool(4); // 3.
16  for(int i = 0; i < num_tasks; i++) {
17      e.submit(new PrintText(i)); // 4.
18  }

```

1. Executor Service needs objects that implement the `Callable<T>` interface (in case we expect a return value) or that implement the `Runnable` interface (in case we don't expect a return value).
2. Since we use the interface `Runnable`, we need to implement the `run()` method. `call()` is the equivalent method we would need to implement if we were using the `Callable` interface.

3. We create the Executor Service thread pool that reuses a fixed number of threads. At any point in this example, at most 4 threads will be active processing tasks. If additional tasks are submitted when all threads are active, they will wait in the queue until a thread is available. If any thread terminates due to a failure during execution prior to shutdown, a new one will take its place if needed to execute subsequent tasks.
4. Tasks can be submitted using `submit()`.

The output of the above example could look something like this:

```

1 Task 0 is executed on 8
2 Task 4 is executed on 10
3 Task 3 is executed on 8
4 Task 2 is executed on 9
5 Task 6 is executed on 11
6 Task 7 is executed on 9
7 Task 5 is executed on 8
8 ...

```

Executor Service is not the right framework for threads that have to wait for each other or are dependent to each other, because we don't have control over when the tasks are scheduled.

4 Concurrency

4.1 Terminology

Useful

The terms wait-free, lock-free, starvation-free, deadlock-free are described in the table of definitions in the beginning. The relationship between these terms looks like this:

	Non-Blocking (no locks)	Blocking (locks)
Everyone makes progress	Wait-free	Starvation-free
Someone makes progress	Lock-free	Deadlock-free

Note: Red arrows in the original image point from 'Wait-free' to 'Lock-free' and from 'Starvation-free' to 'Deadlock-free'.

4.2 Mutual Exclusion

All the below locks are starvation-free and therefore deadlock-free. They are obviously not lock-free and therefore not wait-free (because if they were wait-free they would be lock-free, which they are not). But they are not all fair locks (fairness is defined as "first-come-first-served").

4.2.1 Dekker's Algorithm

Using Dekker's algorithm we can implement mutual exclusion for two processes.

```
1 // process P
2 ... // non critical section
3 wantp = true; // 1.
4 while(wantq) { // 2.
5     if(turn != 1) { // 3.
6         wantp = false; // 4.
7         while(turn != 1) {} // 5.
8         wantp = true; // 6.
9     }
10 }
11 ... // critical section
12 wantp = false; // 7.
13 turn = 2; // 8.
```

Equivalently for process Q:

```
1 // process Q
2 ... // non critical section
3 wantq = true;
4 while(wantp) {
5     if(turn != 2) {
6         wantq = false;
7         while(turn != 2) {}
8         wantq = true;
9     }
10 }
11 ... // critical section
12 wantq = false;
13 turn = 1;
```

1. If process P wants to enter the critical section, it raises its flag by setting `wantp` to `true`.
2. If process Q also raised its flag, then we need to resolve the conflict by entering the while loop. If process Q didn't raise its flag, then process P can enter the critical section immediately.
3. The `turn` variable is shared among both processes. It indicates which process is allowed to enter the critical section. If `turn` is set to 1 then process P can continue. But if `turn` is set to 2 then we have to let process Q go first.
4. If we let process Q go first, we lower our flag.
5. Now we wait until process Q sets `turn` to 2.
6. We raise our flag again and continue to the critical section, because process Q has just left it.
7. Since we left the critical section, we lower our flag.
8. And we set `turn` to 1 such that process Q can enter the critical section if it wants to.

4.2.2 Peterson Lock

Peterson Lock uses the above logic to implement mutual exclusion for two threads.

```
1 public class Peterson {
2     private volatile boolean[] flag = new boolean[2];
3     private volatile int go_last;
4
5     public void lock(int id) { // 1.
6         flag[id] = true; // 2.
7         go_last = id; // 3.
8         while(flag[1-id] && go_last == id) {} // 4.
9     }
10
11     public void unlock(int id) {
12         flag[id] = false; // 5.
13     }
14 }
```

1. The lock method is called with the thread $id \in \{0, 1\}$ as argument.
2. The thread raises its flag.
3. The thread sets the turn variable such that the other thread can enter the critical section first.
4. As long as the other thread wants to access the critical section and the thread marked itself to go last, it executes the while loop (and thereby waits).
5. If the thread finished executing the critical section, it lowers its flag.

The Peterson lock is fair.

4.2.3 Filter Lock

The Filter lock extends the Peterson lock to n threads. It is however not fair.

```
1 public class Filter {
2     private volatile int[] level;
3     private volatile int[] go_last[];
4     private volatile int n;
5
6     public Filter(int n) {
7         this.level = new int[n];
8         this.go_last = new int[n];
9         this.n = n;
10    }
11
12    public void lock(int id) {
13        for(int i = 1; i < n; i++) {
14            level[id] = i;
15            go_last[i] = id;
16            while(others(id, i) && go_last[i] == id) {}
17        }
18    }
19
20    private boolean others(int id, int i) {
21        for(int k = 0; k < n; k++) {
22            if(k != id && level[k] > i) {
23                return true;
24            }
25        }
26        return false;
27    }
28
29    public void unlock(int id) {
30        level[id] = 0;
31    }
32 }
```

Every thread knows its level (`level[id]`). In order to enter the critical section, a thread has to climb up or master all levels. For each level the logic of Peterson's lock is used.

4.2.4 Bakery Lock

The Bakery lock (applied to Switzerland, "post-office lock" would be a more matching name ☺) is built on the intuitive idea of drawing a numbered ticket. Only the thread with the lowest ticket number is allowed to enter the critical section.

```
1 public class Bakery {
2     private volatile boolean[] flag;
3     private volatile int[] ticket;
4     private volatile int n;
5
6     public Bakery(int n) {
7         this.flag = new boolean[n];
8         this.ticket = new int[n];
9         this.n = n;
10    }
11
12    public void lock(int id) {
13        flag[id] = true; // 1.
14        ticket[id] = ticket[0]; // 2.
15        for(int i = 1; i < n; i++) {
16            ticket[id] = Math.max(ticket[id], ticket[i]);
17        }
18        ticket[id]++;
19        while(conflict(id)) {} // 3.
20    }
21
22    private boolean conflict(int id) {
23        for(int i = 0; i < n; i++) {
24            if(i != id && flag[i]) { // 4.
25                if((ticket[i] < ticket[id]) || (ticket[i] == ticket[id]) && (i <
26                    id)) { // 5.
27                    return true;
28                }
29            }
30        }
31        return false;
32    }
33 }
```

1. Since the thread wants to enter the critical section, it raises its flag.
2. In the next few lines we "draw the ticket". This means that we set the ticket to the largest ticket that was already issued plus one.
3. As long there is a thread that wants to enter the critical section and has a lower ticket number, we wait.
4. We iterate over all threads that want to access the critical section (excluding our own thread).
5. If the thread has a lower ticket number, there is a conflict and we return `true`. If the thread has the same ticket number, we only return `true` if the thread has a lower id than our own id.

4.2.5 TAS and CAS

All the above algorithms used $\mathcal{O}(n)$ space (with n being the number of threads). Test-And-Set (TAS) and Compare-And-Swap (CAS) (both atomic read-modify-write operations) allow us to implement mutual exclusion in $\mathcal{O}(1)$ space. The below pseudocode makes the idea behind TAS and CAS clearer.

```

1 // semantics of TAS
2 public boolean TAS(memref a) {
3     if(mem[a] == 0) {
4         mem[a] = 1;
5         return true;
6     }
7     return false;
8 }

```

```

1 // semantics of CAS
2 public int CAS(memref a, int old, int new) {
3     int oldval = mem[a];
4     if(oldval == old) {
5         mem[a] = new;
6     }
7     return oldval;
8 }

```

CAS can be seen as an extension of TAS. Instead of checking for a constant and setting a constant, we instead pass the expected value and the new value. Using TAS and CAS we can build the below spinlocks.

```

1 public class TAS implements Lock {
2     AtomicBoolean s = new AtomicBoolean(false); // 1.
3
4     public void lock() {
5         while(s.getAndSet(true)) {} // 2.
6         // Alternatively: while(!s.compareAndSet(false, true)) {}
7     }
8
9     public void unlock() {
10        s.set(false); // 3.
11    }
12 }

```

1. If the AtomicBoolean variable is set to false then no thread has obtained the lock (the critical section is free). Therefore we initialize the AtomicBoolean variable with **false**.
2. What happens here? By using the `getAndSet()` method, we get the current value of the AtomicBoolean and then immediately set it to the value we specify as argument (in this case **false**). If the AtomicBoolean variable was set to **true**, we would stay in the while loop (which makes sense because this means that another thread is executing code in the critical section). If on the other hand the AtomicBoolean variable was set to **false**, we would exit the while loop and we would set **s** to **true** (indicating that we entered the critical section). Instead of `getAndSet()` we could have also used `compareAndSet()`. The idea is similar: If **s** was set to **false** (the first argument of `compareAndSet()`) we would set it to true (the second argument of `compareAndSet()`) and would get a return value of **true** (indicating that the operation was performed successfully). If on the other hand **s** was set to **true**, we would receive a return value of **false** (indicating that the operation was not performed successfully). If the operation is performed successfully we want to leave the while loop, and when the operation is not performed successfully we want to stay in the while loop. Therefore we have to invert the output of `compareAndSet()` by using **!**.

3. If we want to leave the critical section, we set `s` to `false`. Then another thread that is waiting can set `s` to `true` and thereby enter the critical section.

4.2.6 TATAS

Both versions of the above TAS lock are not really efficient, because they waste computing resources by constantly "get-and-setting" or "compare-and-swapping". In addition they execute these instructions atomically, which is even more resource intensive. Therefore we can use Test-And-Test-And-Set (TATAS).

```
1 public class TATAS implements Lock {
2     AtomicBoolean s = new AtomicBoolean(false);
3
4     public void lock() {
5         do {
6             while(s.get()) {} // 1.
7         } while(s.getAndSet(true)); // 2.
8         // Alternatively: while(!s.compareAndSet(false, true));
9     }
10
11    public void unlock() {
12        s.set(false);
13    }
14 }
```

1. Instead of always trying to Test-And-Set, we first check by only reading, which puts less stress on `s`. As long as `s` is set to `true`, we keep spinning in the while loop.
2. As soon as we read that `s` is set to `false`, we try to set `s` using `getAndSet()` or `compareAndSet()`.

TATAS as an algorithm works, but we cannot expect the Java implementation to work because of some nontrivial interactions in the JMM.

4.2.7 Exponential Backoff

Another way we could decrease contention of `s` is by implementing an exponential backoff: if a check fails, we let the thread go to sleep with exponential duration.

```
1 public class TAS implements Lock {
2     AtomicBoolean s = new AtomicBoolean(false);
3
4     public void lock() {
5         while(s.getAndSet(true)) {
6             // Alternatively: while(!s.compareAndSet(false, true)) {}
7             Thread.sleep(exponentialBackoff); // (pseudo code)
8         }
9     }
10
11    public void unlock() {
12        s.set(false); // 3.
13    }
14 }
```

In terms of performance, the three approaches compare as follows:

4.3 Synchronization

4.3.1 Semaphores

Semaphores can be seen as an extension of locks. It is a tool that allows us to implement barriers or rendezvous. A semaphore is an abstract data type equipped with an integer counter N (that we can set when initializing the semaphore) and the following two atomic operations:

acquire() : blocks until $N > 0$, then decrements N by 1
release() : increments N by 1

This implies:

$N \geq 1 \rightarrow$ unblocked, N threads will be let into the code section
 $N = 0 \rightarrow$ blocked

4.3.2 Rendezvous

We define a rendezvous as a location in the code, where two respective threads must wait for the other to arrive (to "synchronize"). For two threads, the problem can be solved by using two semaphores.

4.3.3 Barriers

A barrier is the same as rendezvous extended to n threads. It can be pictured as a dam (for blocking water). We keep adding threads to the barrier (which blocks them) and once a certain threshold of waiting threads is reached, the dam breaks and all threads continue its operation.

```
1 public class Barrier {
2     Semaphore s = new Semaphore(0);
3     int counter = 0;
4     final int = 10;
5
6     public void await() {
7         synchronized(this) {
8             counter++; // 1.
9         }
10        if(counter == threshold) {
11            s.release(); // 2.
12        }
13        try {
14            s.acquire(); // 3.
15        } catch (InterruptedException e) {}
16        s.release(); // 4.
17    }
18 }
```

1. We increment the counter because a new thread entered the barrier.
2. If enough threads are waiting we call **release()**. This makes N increment by one. Therefore one thread that is waiting in line 18 will be set free and N will be decremented again (per definition of **acquire()**). But the thread that was set free now calls **release()** in line 20, which sets another thread free (and so on). This makes the semaphore drain out over time.
3. Eventually all threads get acquired in this line. Even the one that triggered **release()** in line 15. But as described above, the thread will be set free by another thread that calls **release()** in line 20.

4. Eventually all threads will trigger `release()` in this line.

The above barrier is not reusable, as in the end N is set to 1 (instead of 0). This barrier however is reusable:

```
1 public class Barrier {
2     Semaphore s1 = new Semaphore(0);
3     Semaphore s2 = new Semaphore(1);
4     int counter = 0;
5     final int threshold;
6
7     public Barrier(int threshold) {
8         this.threshold = threshold;
9     }
10
11     public void await() {
12         synchronized(this) {
13             counter++;
14             if(counter == threshold) {
15                 try {
16                     s2.acquire();
17                 } catch (InterruptedException e) {}
18                 s1.release();
19             }
20         }
21         try {
22             s1.acquire();
23         } catch (InterruptedException e) {}
24         s1.release();
25         synchronized(this) {
26             counter--;
27             if(counter == 0) {
28                 try {
29                     s1.acquire();
30                 } catch (InterruptedException e) {}
31                 s2.release();
32             }
33         }
34         try {
35             s2.acquire();
36         } catch (InterruptedException e) {}
37         s2.release();
38     }
39 }
```

4.3.4 Conditions

At the beginning we looked at `wait()` and `notifyAll()` which saves us from using spin locks. `notifyAll()` wakes up all waiting threads that are waiting on the lock on which `notifyAll()` was called on. If we don't want all threads to wake up but only a subset of threads that wait for a certain condition to be true, we can use `condition`.

In this example we wake up only threads whose condition got valid. Image there is a queue in front of a shop on a rainy day and the capacity of people to queue in the line is 10 (any additional person to enqueue to the line would have to stand in the rain).

```
1 public class queue {
2     private LinkedList<Person> queue = new LinkedList<Person>();
3     private final int capacity = 10;
4     private counter = 0;
5     private final Lock lock = new ReentrantLock();
6     private final Condition notEmpty = lock.newCondition();
7     private final Condition notFull = lock.newCondition();
8
9     public void enqueue(Person x) {
10        lock.lock();
11        while(counter == capacity) {
12            try {
13                notFull.await(); // 1.
14            } catch(InterruptedException e) {}
15        }
16        count++;
17        queue.add(x);
18        notEmpty.signalAll(); // 2.
19        lock.unlock();
20    }
21
22    public Person dequeue() {
23        lock.lock();
24        while(counter == 0) {
25            try {
26                notEmpty.await();
27            } catch(InterruptedException e) {}
28        }
29        count--;
30        Person tmp = queue.poll();
31        notFull.signalAll();
32        lock.unlock();
33        return tmp;
34    }
35 }
```

1. Instead of calling `wait()` we let the thread "await" to the condition `notFull`, so that we can wake it up once the queue is not full anymore.
2. Since we added one person to the queue, we know that it is not empty. Therefore we wake up all threads that listen to the condition `notEmpty`.

4.4 Lock Granularity

In the following 4 sub chapters, the locking strategies coarse grained, fine grained, optimistic, and lazy are explained. We use a sorted linked list as an example, but with some customization they can be applied to many different concurrent data structures.

For our example the following holds:

- The linked list contains `Node` objects which only have the attributes `next` (of type `Node`) and `key` (of type `int`). The integer values `key` of all nodes are sorted.
- `head` is the first element of the list.
- We need to implement `remove(x)` which removes integer `x` from the list. It returns `false` if the list does not contain `x`.
- We need to implement `contains(x)` which returns `true` if the list contains integer `x`.

At the end all 4 approaches are compared with each other.

4.4.1 Coarse Grained Locking

The idea behind coarse grained locking is to lock the entire data structure.

```
1 // private Node head is defined
2 // private Lock lock is defined
3 public boolean remove(int x) {
4     lock.lock(); // lock entire data structure
5     Node pred, curr;
6     try {
7         pred = head;
8         curr = pred.next;
9         while(curr.key < x) { // we iterate over list
10             pred = curr;
11             curr = curr.next;
12         }
13         if(curr.key == x) { // if x was found, remove it
14             pred.next = curr.next;
15             return true;
16         }
17         return false;
18     } finally { // before returning, we unlock data structure
19         lock.unlock();
20     }
21 }
22
23 public boolean contains(int x) {
24     lock.lock();
25     Node node = head;
26     try {
27         while(node.key < x) {
28             node = node.next;
29         }
30         return node.key == x;
31     } finally {
32         lock.unlock();
33     }
34 }
```

4.4.2 Fine Grained Locking

The idea behind fine grained locking is to always have a lock on the previous (`pred`) and current (`curr`) node. This allows multiple threads to operate on the data structure.

```
1  // private Node head is defined
2  // private Lock lock is defined
3  public boolean remove(int x) {
4      Node pred = head;
5      pred.lock();
6      try {
7          Node curr = pred.next;
8          curr.lock();
9          try {
10             while(curr.key < x) {
11                 pred.unlock(); // unlock previous
12                 pred = curr; // set previous to current
13                 curr = curr.next; // set current to next element
14                 curr.lock(); // lock current
15             }
16             if(current.key == key) {
17                 pred.next = curr.next; // remove element
18                 return true;
19             }
20             return false;
21         } finally {
22             curr.unlock();
23         }
24     } finally {
25         pred.unlock();
26     }
27 }
28
29 public boolean contains(int x) {
30     Node pred = head;
31     pred.lock();
32     try {
33         Node curr = pred.next;
34         try {
35             while(curr.key < x){
36                 pred.unlock;
37                 pred = curr;
38                 curr = curr.next;
39                 curr.lock;
40             }
41             return curr.key == x;
42         } finally {
43             curr.unlock();
44         }
45     } finally {
46         pred.unlock();
47     }
48 }
```

At any point of time, `curr` and its predecessor `pred` are locked.

4.4.3 Optimistic Locking

In order to decrease the amount of locking and unlocking operations, we come up with optimistic locking. We don't lock, but we validate before deleting. If validation fails, we simply try again.

```
1 // private Node head is defined
2 // private Lock lock is defined
3 public boolean remove(int x) {
4     while(true) { // retry if validation fails
5         Node pred = head;
6         Node curr = pred.next;
7         while(curr.key < key) { // observe that we don't lock
8             pred = curr;
9             curr = curr.next;
10        }
11        pred.lock(); // now we lock both elements
12        curr.lock();
13        try {
14            if(validate(pred, curr)) {
15                if(curr.key == x) {
16                    pred.next = curr.next;
17                    return true;
18                }
19                return false;
20            }
21        } finally {
22            pred.unlock();
23            curr.unlock();
24        }
25    }
26
27    private boolean validate(Node pred, Node curr) {
28        Node node = head;
29        while(node.key <= pred.key) {
30            if(node == pred) {
31                return node.next == curr; // check if successor of pred is curr
32            }
33            node = node.next;
34        }
35        return false;
36    }
37
38    public boolean contains(int x) {
39        Node node = head;
40        while(node.key < x) {
41            node = node.next;
42        }
43        return node.key == x;
44    }
```

4.4.4 Lazy Locking

Lazy locking is similar to optimistic locking, but by introducing a new boolean attribute `marked` we can increase the performance of the `validate` method. If `marked` is set to `true`, the node is logically deleted.

```
1 // private Node head is defined
2 // private Lock lock is defined
3 public boolean remove(int x) {
4     while(true) {
5         Node pred = head;
6         Node curr = pred.next;
7         while(curr.key < x) {
8             pred = curr;
9             curr = curr.next;
10        }
11        pred.lock();
12        curr.lock();
13        try {
14            if(validate(pred, curr)) {
15                if(curr.key == x) {
16                    curr.marked = true // set marked attribute to true
17                    pred.next = curr.next;
18                    return true;
19                }
20                return false;
21            }
22        } finally {
23            pred.unlock();
24            curr.unlock();
25        }
26    }
27 }
28
29 public boolean validate(Node pred, Node curr) {
30     return !pred.marked && !curr.marked && pred.next == curr; // check if both
31     nodes are not marked and if successor of pred is curr
32 }
33
34 public boolean contains(int x) {
35     Node node = head;
36     while(curr.key < x) {
37         node = node.next;
38     }
39     return node.key == x && !node.marked; // check if node is not marked
40 }
```

4.4.5 Pros and Cons

Useful

Up and downsides of locking strategies:

- | | |
|--------------------------------|--|
| Coarse Grained locking: | + Easy to implement |
| | – Only one thread can operate on data structure |
| Fine Grained locking: | + Multiple threads can operate on data structure |
| | – The slowest thread sets the tempo (no overtaking) |
| | – Many lock and unlock operations |
| Optimistic locking: | + Fast |
| | + No blocking of other threads |
| | – More complicated implementation |
| | – Multiple while loop iterations on data structures with high contention |
| Lazy locking: | + Faster validate method |
| | – Additional marked attribute required |

4.5 Non Blocking Algorithms

Locks allow us to make operations appear to be atomic, but locking also comes with some downsides.

- Locks are pessimistic by design (assume the worst and implement mutual exclusion).
- Locks come with performance issues (locking and unlocking requires reading from main memory).
- If one thread is delayed in the critical section, all threads suffer. Additionally there is a possibility for deadlocks to occur.

Therefore we are interested in approaches that are lock-free (for instance by using `testAndSet()` or `compareAndSet()`). One of the problems that can come up on lock-free concurrent data structures is the ABA problem.

4.5.1 ABA Problem

Let's consider this implementation of a lock-free stack's `pop()` function:

```
1 public Integer pop() {
2     Node head, next;
3     do {
4         head = stack.get(); // get top element in stack
5         if(head == null) { // if stack is empty return null
6             return null;
7         }
8         next = head.next;
9     } while(!stack.compareAndSet(head, next)); // use CAS to set head to next
10    return head.item;
11 }
```

However we could observe the following scenario: Assume thread A executes `pop()` and `head` gets set to `a` (the top element in the stack, line 4). Then in line 6 `next` gets set to `b` (the element after `a`). Then thread A wants to call `compareAndSet(a, b)` in line 8. But it may be that before `compareAndSet()` is executed, another thread B popped elements `a` and `b` from the stack and inserted elements `c` and `a` to the stack (so

the two top most elements now are **a** and **c**). When thread A is about to execute `compareAndSet(a, b)` the CAS operation will be successfully triggered (since **a** still is the top most element in the stack) but it will set the new top most element to **b** (which just got deleted by thread B). This was an example of the ABA problem.

Definition

ABA problem: The ABA problem occurs when a thread fails to recognize that a (single) memory location was modified by another thread and thereby mistakenly assumes that the overall state has not been changed.

There are solutions to the ABA problem:

- **Double CAS (DCAS):** Instead of using the default CAS operation, we can check whether both `head` and `head.next` are still in the expected state. However, DCAS is not implemented in most of the computer architectures.
- **Hazard Pointers:** Before some thread A wants to read or modify an object, it puts it into the hazard array (and thereby making it hazardous). When another thread B now tries to access the object, it first checks if it is present in the hazard array. If it is present in the array, then it waits. Once thread A finished operating on the object, it removes the object from the hazard array.
- **Pointer Tagging:** A few bits (n) of the pointer are reserved for a tag. The number stored in the tag gets incremented by 1 every time the object is placed into a new data structure. This makes the ABA problem much less likely to happen because now 2^n versions of the pointer exist. There still is a chance that the ABA problem occurs: when the pointer is used 2^n times in between accesses of thread A, the number stored in the tag will be the same as in the beginning.

Transactional Memory and the garbage collector can also be used to solve the ABA problem. Garbage collector however is very slow.

4.6 Consensus

In this context, the idea behind consensus is that we want to make multiple threads agree to one value. There are three requirements that a correct consensus protocol must fulfill:

Definition

In order for a consensus protocol to be correct, it has to be:

- **wait-free:** A consensus must be found in a finite number of time. Wait-free implies lock-free, which means that we cannot use `synchronized()` or the previously mentioned locks to implement the consensus protocol.
- **consistent:** All threads need to agree to the same value.
- **valid:** The consensus must be one thread's input. If this requirement was not present, then we could just return some arbitrary value for all threads.

Below is an example of a correct consensus protocol.

```
1 AtomicInteger consensus = new AtomicInteger(Integer.MAX_VALUE);
2
3 // we assume no thread's input is Integer.MAX_VALUE
4 public Integer consensus(Integer input) {
5     // the first thread sets the consensus value. Since no thread's input is
6     // Integer.MAX_VALUE, the CAS operation always fails
7     consensus.compareAndSet(Integer.MAX_VALUE, input);
8     return consensus.get();
9 }
```

5 Additional Topics

5.1 Message Passing Interface (MPI)

Up until now we looked at different ways of dealing with multiple threads that want to access the same data set concurrently. We can use the Fork/Join framework to partition the data set or use locks to make sure only one thread is able to operate on some parts of the data set. But using Message Passing, we can let the threads run in isolation and only communicate via messages which must be explicitly received. MP allows us to benefit from the upsides of distributed memory systems (and is primarily used for distributed memory systems).

Useful

Shared memory: + Easy to use

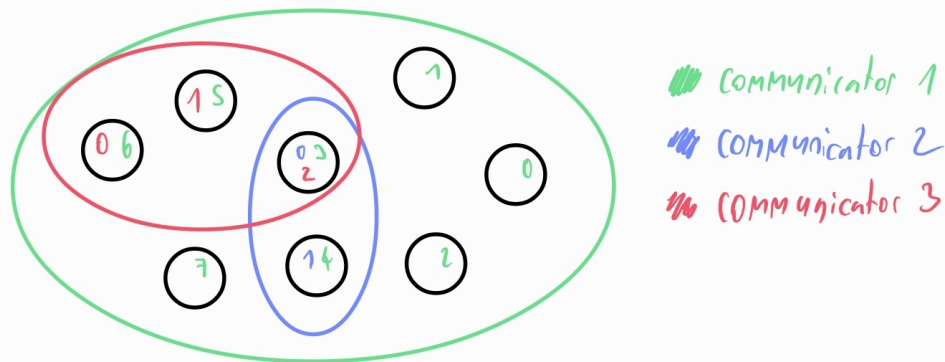
- High coherence overhead
- Limited scalability

Distributed memory – Harder to use

- + No coherence overhead (memory is shared by sending / receiving)
- + High scalability

The Message Passing Interface (MPI) is a portable message-passing standard that is not bound to a particular programming language. Further we focus on MPJ (Message Passing in Java).

- MPI workers (threads) are organized into communicators and groups (communicators are created from groups).
- Workers can be in multiple communicators.
- For each communicator a worker is part of, it has a unique rank (identifier). If workers are in n communicators, they have n ranks (one rank per communicator).
- Workers can communicate inside communicators - e.g. a broadcast from one worker would be sent to all workers in the same communicator.



Some important methods:

```

1 MPI.Init(); // initializes the MPI library
2 MPI.COMM.Size(); // returns amount of workers there are in the communicator COMM
3 MPI.COMM.Rank(); // returns the rank of the worker (which executed this method) in
  communicator COMM
4 MPI.Finalize(); // Clean up all MPI state (last call)

```

How can we send and receive data across workers? We distinguish between point to point communication (one worker communicates with another worker - 1 : 1 relationship) and collective communication (one worker communicates with n workers - 1 : n or n : 1 relationship). The most important MPI functions are listed below.

Definition

Point to point:

Send() sends the buffer to one other worker.

```

MPI.COMM.Send(
    Object buf,           // pointer to array to be sent
    int offset,           // offset within array
    int count,            // number of elements to be sent
    Datatype datatype,    // data type of elements
    int dest,             // destination worker rank
    int tag               // data id tag
)

```

Recv() receives the buffer.

```

MPI.COMM.Recv(
    Object buf,           // pointer to array where data should be stored
    int offset,           // offset within array
    int count,            // number of elements to be received
    Datatype datatype,    // data type of elements
    int source,           // source worker rank
    int tag               // data id tag
)

```

Collective:

Bcast() broadcasts the buffer to all workers that are in the same communicator as the sender.

```

MPI.COMM.Bcast(
    Object buf,           // buffer array
    int offset,           // offset in buffer

```

```

    int count,           // number of elements in buffer
    Datatype datatype,   // data type of elements in buffer
    int root              // rank of broadcast worker
)

```

Reduce() gathers values distributed on multiple workers and computes them using an associative operator (like MPI.SUM).

```

MPI.COMM.Reduce(
    Object sendbuf,       // send buffer array
    int offset,           // offset in send buffer
    Object recvbuf,       // receive buffer array
    int recvoffset,       // offset in receive buffer
    int count,            // number of elements in receive buffer
    Datatype datatype,    // data type of elements in send buffer
    Op op,                // associative operator
    int root              // rank of root worker
)

```

Allreduce() is equivalent to Reduce(), but at the end the calculated value is available to all workers.

```

MPI.COMM.Allreduce(
    Object sendbuf,       // send buffer array
    int offset,           // offset in send buffer
    Object recvbuf,       // receive buffer array
    int recvoffset,       // offset in receive buffer
    int count,            // number of elements in receive buffer
    Datatype datatype,    // data type of elements in send buffer
    Op op,                // associative operator
)

```

Scatter() evenly distributes the buffer's elements to all workers.

```

MPI.COMM.Scatter(
    Object sendbuf,       // send buffer array
    int sendoffset,       // offset in send buffer
    int sendcount,        // number of elements to send
    Datatype sendtype,    // data type of elements in send buffer
    Object recvbuf,       // receive buffer array
    int recvoffset,       // offset in receive buffer
    int recvcount,        // number of elements to receive
    Datatype recvtype,    // data type of elements in receive buffer
    int root              // rank of receiving worker
)

```

Gather() can be pictured as the inverse function of Scatter(). It retrieves the elements and places them inside the buffer of the worker specified in argument root.

```

MPI.COMM.Gather(
    Object sendbuf,       // send buffer array
    int sendoffset,       // offset in send buffer
    int sendcount,        // number of elements to send
    Datatype sendtype,    // data type of elements in send buffer
    Object recvbuf,       // receive buffer array
    int recvoffset,       // offset in receive buffer
    int recvcount,        // number of elements to receive
    Datatype recvtype,    // data type of elements in receive buffer
    int root              // rank of receiving worker
)

```

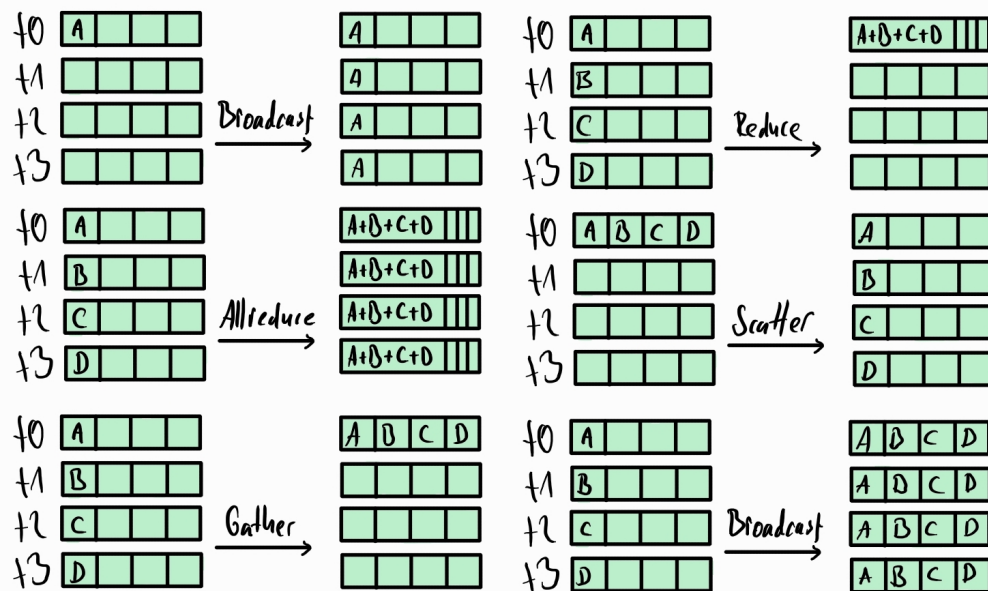
```
)
```

Allgather() is equivalent to Gather(), but in the end the retrieved values are available to all workers.

```
MPI.COMM.Allgather(  
    Object sendbuf,      // send buffer array  
    int sendoffset,      // offset in send buffer  
    int sendcount,       // number of elements to send  
    Datatype sendtype,   // data type of elements in send buffer  
    Object rcvbuf,       // receive buffer array  
    int rcvoffset,       // offset in receive buffer  
    int rcvcount,        // number of elements to receive  
    Datatype rcvtype,    // data type of elements in receive buffer  
    int root             // rank of receiving worker  
)
```

There are many more functions.

The above functions can be visualized like that:



Here is an example of how MPI can be used in practice. The following code normalizes a vector x to its unit vector $\hat{x} = \frac{x}{\|x\|}$ with $x = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$.

```
public static void Normalize(float x[]) {
    MPI.Init();
    int mpiRank = MPI.COMM_WORLD.Rank();
    int mpiSize = MPI.COMM_WORLD.Size();
    // assuming mpiSize evenly divides the number of elements in the vector
    int xLocalSize = x.length / mpiSize;
    float xLocal[] = new float[xLocalSize];

    // worker 0 scatters to all other workers
    MPI.COMM_WORLD.Scatter(x, 0, xLocalSize, MPI.FLOAT, xLocal, 0, xLocalSize, MPI
        .FLOAT, 0);

    // each worker calculates its part's sum
    float xLocalSquared[] = {0};
    for(int i = 0; i < xLocalSize; i++) {
        xLocalSquared += xLocal[i] * xLocal[i];
    }
    float xSquared[] = {0};

    // sums up all results and makes it available to all workers
    MPI.COMM_WORLD.Allreduce(xLocalSquared, 0, xSquared, 0, 1, MPI.FLOAT, MPI.SUM)
        ;

    // divides every element by vector's length
    float factor = 1f / ((float) Math.sqrt(xSquared[0]));
    for(int i = 0; i < xLocalSize; i++) {
        xLocal[i] *= factor;
    }

    // Gather all results to worker 0
    MPI.COMM_WORLD.Gather(xLocal, 0, xLocalSize, MPI.FLOAT, x, 0, xLocalSize, MPI.
        FLOAT, 0);
}
```

There are blocking and non-blocking operations. `Send()` and `Recv()` for instance are blocking, meaning that they won't return until the action has been completed locally (the message has been fully copied to the outgoing buffer / the message has been fully copied from the incoming buffer). Their non-blocking counterparts `irecv()` and `isend()` return immediately.

There are synchronous and asynchronous messages. `SSend()` for instance is the synchronous version of `Send()`. It waits until the message has been received by the receiving worker. `Send()` on the other hand places the message inside a buffer and doesn't wait until the receiver picked up the message (fire and forget).

5.2 Transactional Memory

The goal of transactional memory is to remove the burden of synchronization away from the programmer and to to the system (hard or software). As an example, the ideal way of programming would be:

```
atomic {
    a.withdraw(amount);
    b.deposit(amount);
}
```

This is the same idea behind locks, but as mentioned locks come with their own downsides. The upsides of transactional memory are:

- higher level semantics (as seen in above example)
- composable (unlike locks)
- optimistic by design (does not require mutual exclusion)

Useful

Two important properties of transactional memory (TM) are:

- **Atomicity:** Other threads either see the initial state before the transaction or the state after the transaction. The transaction appears to be executed in a single, uninterruptible step.
- **Isolation:** During a transaction's execution, changes of other threads are not observed. Before executing the transaction, TM takes a snapshot of the relevant parts of the system and operates on them. In the end TM commits its changes.

TM is related to database transactions (and their properties "ACID" - Atomicity, Consistency, Isolation, Durability), but consistency and durability are not really applicable to TM.

TM can cause some problems:

1. transaction A takes snapshot of system (let variable a be 1 in the snapshot).
2. transaction B that was already running now commits and changes a to 2.
3. since transaction A runs in isolation, it doesn't notice the change of variable a . Let the code that transaction A executes be dependent on value a .
4. transaction A finishes and commits its changes to the system. But it operated on the wrong value of a , which makes the operation invalid.

Transaction A has to be aborted. These issues are handled by a concurrency control mechanism.

5.3 Histories

We first define invocations and responses.

Definition

Invocation: Trigger of a method call, where a result (response) is expected. Typical representation of an invocation: $\langle thread \rangle \langle object \rangle . \langle method \rangle (\langle arguments \rangle)$, for instance $A \ o.enq(x)$ (on thread A , we call method enq on object o with argument x).

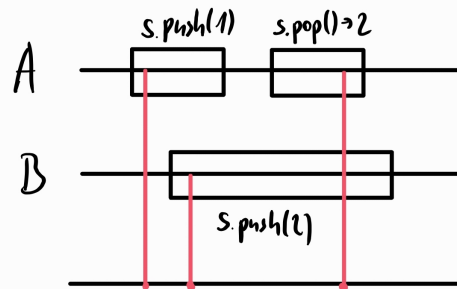
Response: Returns result of invocation. Typical representation of a response: $\langle thread \rangle \langle object \rangle : \langle result \rangle$, for instance $A \ o:2$ (on thread A object o received output 2).

History: A history is a sequence of invocations and responses.

Below is an example of a history.

```
// s is a stack
A s.push(1)
B s.push(2)
A s:void
A s.pop()
A s:2
B s:void
```

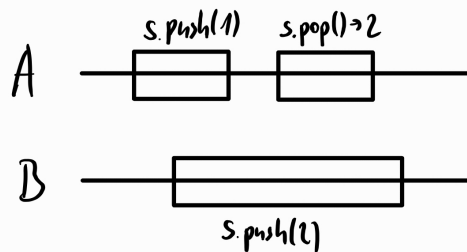
We can visualize the history as shown below.



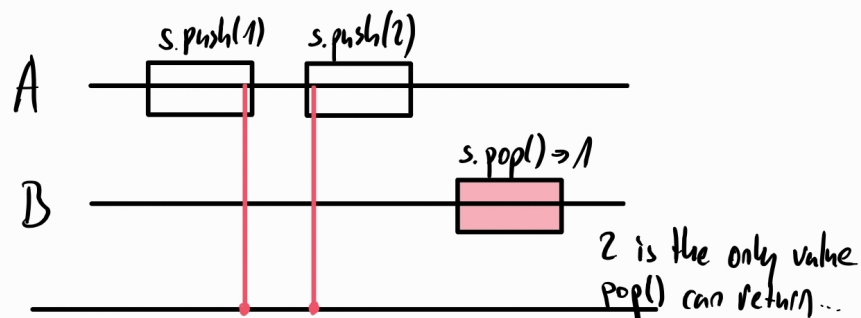
We can find invocation points such that the history "makes sense", and therefore we can say that the history is linearizable, and therefore also sequentially consistent (linearizability and sequential consistency are defined later).

If we want to check if a history is sequentially consistent, we can ignore the order between events across threads. Example:

```
// s is a stack
A s.push(1)
A s:void
A s.push(2)
A s:void
B s.pop()
B s:1
```



We cannot find linearization points, so the history is not linearizable. However it is sequentially consistent as we can reorder the history as shown below (we can ignore the order between events across threads).



Now the history makes sense.

Useful

Linearizability requires both the order between events across threads and the order between thread local events to be preserved.

Sequential consistency only requires the thread local event order to be preserved.

Some further definitions:

Definition

Thread projection: We only consider invocations and responses of a specific thread. Example: If H is the history and we only care about invocations and responses of thread B , then we denote $H|B$ as the projection of thread B .

```
// s is a stack
// we only care about invocations and responses of thread B
// A s.push(1)
B s.push(2)
// A s:void
// A s.pop()
// A s:2
B s.void
```

Object projection: Similar to thread projection, but we only consider invocations and responses of a specific object.

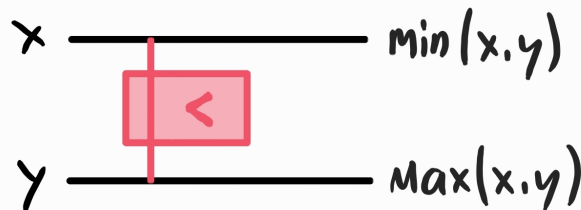
Complete history: A history without pending invocations (every invocation has a matching response)

Sequential history: Consists only of non-interleaving method calls.

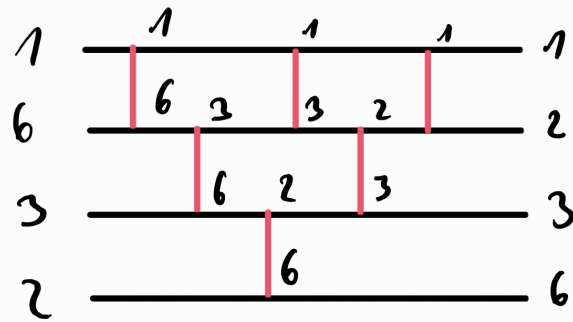
Equivalence: Two histories are equivalent if all of their thread projections are identical.

5.4 Sorting Networks

Sorting networks are a sequence of comparisons. A comparison takes the number at an index and compares it to the number at another index. It then swaps the two numbers if the number with higher index is smaller than the number with lower index. Example:



One comparison operation can be described as follows.



In order to prove if a sorting network is valid, we can make use of the 0-1-principle.

Useful

If a network with n input lines sorts all 2^n sequences of zeros and ones, it will sort any arbitrary sequence of n numbers. Example:

